TP 2 — Détection du sarcasme en arabe : déséquilibre, multi-tâche et généralisation

ArSarcasm-v2 · PLM zero-shot · embeddings statiques + réseaux profonds · PLM gelé et fine-tuné (mono- et multi-tâche) · tests de généralisation

Binôme ou individuel · Rendu : dépôt Git + compte-rendu (dernières pages)

1. Objectifs

- Construire un **prétraitement de l'arabe** correct, et vérifier qu'il ne **détruit pas les indices du sarcasme** (emojis, allongements, ponctuation, rires).
- Détecter le **sarcasme** (classes déséquilibrées) avec 5 familles d'approches : baseline TF-IDF, **PLM utilisé directement** (zero-shot), **embeddings statiques + réseaux profonds**, **PLM gelé** + tête profonde, **PLM fine-tuné**.
- Gérer le **déséquilibre** (pondération, ré-échantillonnage, seuil) et choisir la bonne métrique.
- Exploiter les autres annotations par **apprentissage multi-tâche** (sarcasme + sentiment) sur un encodeur partagé — lien direct avec le cours sur l'encodeur.
- Évaluer **comme en production** : test officiel jamais vu, décalage de prior, dialecte non vu, raccourcis (shortcuts), analyse d'erreurs.

2. Prérequis & environnement

- Cours 1 (embeddings), Cours 2 (attention), Cours 3 (bloc Transformer, encodeur, BERT).
- Python 3.10+ ; `torch`, `transformers`, `datasets`, `evaluate`, `scikit-learn`, `gensim`, `pyarabic`, `emoji` ; préprocesseur AraBERT (`pip install arabert`, ou `preprocess.py` du dépôt aub-mind/arabert).
- GPU : Colab ou Kaggle (une T4 suffit). Environnement géré avec uv (ou `requirements.txt` figé), dépôt Git créé dès la première séance.

3. Données — ArSarcasm-v2

Corpus de **15 548 tweets arabes** publié pour la tâche partagée WANLP 2021 (sarcasme et sentiment). Chaque tweet porte **trois annotations** : sentiment, sarcasme et dialecte. Le découpage train/test officiel est fourni et doit être respecté.

Élément	Détail
Source	github.com/iabufarha/ArSarcasm-v2 (dossier `ArSarcasm-v2/`)
Fichiers	`training_data.csv` (12 548 tweets) · `testing_data.csv` (3 000 tweets)
Colonnes	`tweet` · `sarcasm` (TRUE / FALSE) · `sentiment` (NEG / NEU / POS) · `dialect` (msa, egypt, gulf, levant, magreb)
Tâche du TP	Prédire la colonne `sarcasm` (binaire) à partir de `tweet`. `sentiment` et `dialect` servent de tâches auxiliaires (Partie E) et de sous-groupes d'évaluation — **jamais comme entrées du modèle**.
Métriques	**F1 de la classe sarcastique** (principale, métrique officielle — meilleur score publié : 0,6225) ; **PR-AUC** ; macro-F1. L'accuracy n'est **pas** une métrique acceptable ici (voir Q1).

Protocole imposé (identique pour tout le monde)

```
import pandas as pd
from sklearn.model_selection import train_test_split

BASE = "https://raw.githubusercontent.com/iabufarha/ArSarcasm-v2/main/ArSarcasm-v2/"
full = pd.read_csv(BASE + "training_data.csv")
test = pd.read_csv(BASE + "testing_data.csv")     # scellé jusqu'à la Partie F
```

```
full["id"], test["id"] = range(len(full)), range(len(test))

train, val = train_test_split(full, test_size=0.10, random_state=42,
                              stratify=full["sarcasm"])
```

Règles du jeu

• Toutes les décisions (prétraitement, architecture, hyperparamètres, nombre d'époques, **seuil de décision**) se prennent sur **val**. Le test officiel ne sert qu'aux modèles finaux, en Partie F.

• Toute transformation apprise (vocabulaire, TF-IDF, mise à l'échelle, poids de classes, ré-échantillonnage) est ajustée sur **train uniquement**.

• Chaque exécution est journalisée dans `results/results.csv` : modèle, prétraitement, graine, métriques val/test, temps d'entraînement, nombre de paramètres entraînés.

4. Prétraitement — à faire « comme il se doit »

Le prétraitement n'est pas une étape unique : **il dépend du modèle**. Un bon prétraitement pour TF-IDF peut dégrader un PLM, et inversement. Il doit être **audité, justifié, testé et mesuré**.

4.1 Audit du texte brut (avant toute normalisation)

- Affichez 30 tweets bruts de train et 30 de test. Vous trouverez **au moins un artefact de format** (introduit par l'export CSV, pas par l'auteur du tweet) : identifiez-le, mesurez sa fréquence dans train **et** dans test, et sa corrélation avec les labels.
- Comptez : URL, mentions @, hashtags, emojis, rires (هههه), ponctuation répétée, diacritiques, tatweel, lettres répétées, caractères latins, chiffres indiens (٠١٢).
- Doublons : exacts **et** après normalisation, dans train et entre train et test. Retirez de train les tweets présents dans test et rapportez leur nombre. Repérez les doublons aux **labels contradictoires** et décidez d'une politique (justifiée).

5. Déroulé

Partie A — Exploration, raccourcis & baseline

- Taux de sarcasme dans train et dans test, par dialecte et par sentiment.
- Classifieur trivial « jamais sarcastique » : accuracy et F1 sur val. Conclusion sur l'accuracy ?
- Baseline : TF-IDF (mots 1–2 + caractères 2–5) + régression logistique `class_weight="balanced"` ; **seuil de décision choisi sur val** (maximisation du F1 sarcastique).
- **Audit des raccourcis** : taux de sarcasme selon la présence d'une URL, d'une mention, de l'artefact de format (4.1). Évaluez un « classifieur à une seule règle » pour chacun : que vaut-il ?

Partie B — PLM utilisé directement (aucun entraînement)

- **B1. Zero-shot par NLI** : `MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7`. Testez **au moins 3 formulations** d'hypothèse (arabe, français, anglais). Le score P(sarcastique) est **seuillé sur val** : même sans entraînement, la validation reste indispensable.
- **B2. Transfert par heuristique** : modèle de sentiment déjà fine-tuné `CAMeL-Lab/bert-base-arabic-camelbert-da-sentiment` + règle « sentiment négatif ⇒ sarcastique ». Inspectez `model.config.id2label`.

```
from transformers import pipeline
zs = pipeline("zero-shot-classification",
              model="MoritzLaurer/mDeBERTa-v3-base-xnli-multilingual-nli-2mil7")
labels = ["ساخرة",     # sarcastique
          "جادة"]      # sérieuse
template = "هذه التغريدة {}."   # « Ce tweet est {}. »
out = zs(tweet, candidate_labels=labels, hypothesis_template=template)

sa = pipeline("text-classification",
              model="CAMeL-Lab/bert-base-arabic-camelbert-da-sentiment")
print(sa.model.config.id2label)
```

Piège — contamination

• Le modèle de B2 a été fine-tuné sur ASTD, ArSAS et SemEval. Or ArSarcasm (v1), inclus dans ArSarcasm-v2, a été construit en **ré-annotant ASTD et SemEval 2017**.

• Il a donc peut-être déjà vu certains de vos tweets de test. Gardez-le en tête en interprétant B2.

Partie C — Embeddings statiques + réseaux profonds (PyTorch)

- Tokenisation en mots sur le texte normalisé ; vocabulaire construit sur train (fréquence ≥ 2) ; **taux d'OOV** sur val et test.
- Couche d'embedding : (i) aléatoire ; (ii) **AraVec** (Twitter, unigrammes, 100 ou 300 dim.), figée puis dégelée. Appliquez `clean_str` d'AraVec avant la recherche dans son vocabulaire.
- Architectures — même budget d'entraînement, early stopping sur val F1-sarcastique :
  - **C1.** BiLSTM (dernier état caché ou max-pooling).
  - **C2.** BiLSTM + **pooling attentionnel** implémenté par vous : $\alpha_i = \text{softmax}(v^T \tanh(W h_i))$, $c = \Sigma\, \alpha_i\, h_i$.
  - **C3. Mini-encodeur Transformer** : 2 couches, d_model = 128, 4 têtes, encodage positionnel sinusoïdal, masque de padding. Ablation : **sans** encodage positionnel.
  - **C4.** Optionnel **CNN sur caractères** (char-CNN) : capte-t-il mieux allongements, rires et emojis ?
- **Déséquilibre** — sur le meilleur modèle de C1–C3, comparez : (a) perte pondérée ; (b) sur-échantillonnage de la classe minoritaire (**dans train uniquement**) ; (c) perte standard + seuil optimisé sur val. Optionnel : focal loss.

Partie D — PLM gelé : l'encodeur comme extracteur de représentations

- **D1.** `aubmindlab/bert-base-arabertv02-twitter` gelé : [CLS] vs **moyenne des tokens** (pondérée par le masque d'attention) → régression logistique (seuil sur val).
- **D2.** États de **tous les tokens** (encodeur gelé) → tête BiLSTM + attention de C2. **Visualisez l'attention sur 5 tweets sarcastiques.**

Partie E — Fine-tuning : mono-tâche vs multi-tâche

- Deux PLM pré-entraînés sur des tweets : `aubmindlab/bert-base-arabertv02-twitter` et `UBC-NLP/MARBERTv2`.
- **E1. Mono-tâche** (sarcasme) : perte pondérée, seuil choisi sur val.
- **E2. Multi-tâche** : encodeur partagé + 2 têtes (sarcasme, sentiment) ; perte $L = L_{sarc} + \lambda \cdot L_{sent}$, avec $\lambda \in \{0{,}3 ; 0{,}5 ; 1\}$ choisi sur val. Optionnel : 3e tête « dialecte ».
- Point de départ : lr 2e-5, batch 32, 3 à 5 époques, warmup 10 %, weight decay 0,01, fp16, early stopping sur val F1-sarcastique. **3 graines** par modèle final (moyenne ± écart-type).

```
class MultiTask(nn.Module):
    def __init__(self, name):
        super().__init__()
        self.enc = AutoModel.from_pretrained(name)
        h = self.enc.config.hidden_size
        self.sarc, self.sent = nn.Linear(h, 2), nn.Linear(h, 3)

    def forward(self, input_ids, attention_mask):
        out = self.enc(input_ids=input_ids, attention_mask=attention_mask)
        cls = out.last_hidden_state[:, 0]
        return self.sarc(cls), self.sent(cls)

# loss = ce_sarc(logits_sarc, y_sarc) + lam * ce_sent(logits_sent, y_sent)
```

Partie F — Tests de généralisation (le test officiel est « descellé »)

Appliquez F1 à F6 à la baseline et aux meilleurs modèles de C, D, E1 et E2.

- **F1. Décalage de prior** : comparez le taux de sarcasme de val et de test. Avec le seuil choisi sur val, quel F1 sur le test ? **A posteriori seulement** (analyse, pas sélection) : tracez F1 = f(seuil) sur le test.
- **F2. Dialecte non vu** : retirez tous les tweets egypt de train, réentraînez votre meilleur modèle E, évaluez sur les tweets egypt du test ; comparez au même modèle entraîné sur tout train.
- **F3. Raccourcis** : scores sur les sous-ensembles du test avec / sans URL. Réentraînez en supprimant les jetons URL et mention : qu'est-ce qui change ?

- **F4. Indices du sarcasme** : sur les tweets sarcastiques correctement détectés, supprimez emojis, rires et ponctuation répétée ; mesurez la **chute du taux de détection**.
- **F5. Significativité** : 3 graines, IC à 95 % par bootstrap (1 000 rééchantillonnages), test de McNemar entre vos deux meilleurs modèles.
- **F6. Analyse d'erreurs** : 30 erreurs (15 faux négatifs, 15 faux positifs) classées en catégories — connaissance du monde / contexte requis, ironie sans marqueur, marqueur mal interprété, dialecte, **bruit d'annotation**. Croisez avec les doublons aux labels contradictoires trouvés en 4.1.

Partie G — Comparaison & décision

- Tableau final : qualité, robustesse, temps d'entraînement, **latence d'inférence par tweet sur CPU**, taille du modèle, test de généralisation et performances.
- Matrice de confusion du modèle retenu (au seuil retenu).

6. Questions (à traiter dans le compte-rendu)

- **Q1.** Taux de sarcasme dans train, val et test, globalement et par dialecte. Accuracy et F1 du classifieur « jamais sarcastique ». Quelle métrique retenez-vous, et pourquoi ?
- **Q2.** Raccourcis : quels indices superficiels (artefact de format, URL, mentions) sont corrélés au label ? Que vaut un classifieur à une seule règle ? Qu'en concluez-vous sur la validité de vos scores ?
- **Q3.** Quels choix de prétraitement avez-vous faits par famille de modèles ? Lesquels détruisent des indices du sarcasme (ablation 4.3, chiffres à l'appui) ?
- **Q4.** Zero-shot (B1) : score obtenu et sensibilité à la formulation de l'hypothèse (arabe / français / anglais). Pourquoi un modèle NLI peut-il classer sans exemple d'entraînement ? Quel rôle joue l'attention entre prémisse et hypothèse ?
- **Q5.** Heuristique « négatif ⇒ sarcastique » (B2) : précision, rappel, F1. Que révèle-t-elle de la relation entre sarcasme et sentiment dans ce corpus ? Son score est-il exempt de contamination ?
- **Q6.** Déséquilibre : comparez pondération de la perte, sur-échantillonnage et optimisation du seuil (tableau). Pourquoi le sur-échantillonnage doit-il être fait après la séparation train / val ?
- **Q7.** C1 vs C2 vs C3 (et C4) : résultats. Les poids d'attention se concentrent-ils sur les marqueurs attendus (emojis, rires, allongements, ponctuation) ? Effet de l'encodage positionnel ?
- **Q8.** PLM gelé (D) vs fine-tuné (E1) : gain et coût (temps, paramètres entraînés). AraBERT-twitter vs MARBERT : lequel, et pourquoi ?
- **Q9.** Multi-tâche (E2) : l'ajout du sentiment améliore-t-il la détection du sarcasme ? Effet de λ. Pourquoi une tâche auxiliaire peut-elle aider… ou nuire ?
- **Q10.** Décalage de prior (F1) : écart entre le seuil choisi sur val et le seuil optimal a posteriori sur le test. Que mettriez-vous en place en production (recalibration, monitoring) ?
- **Q11.** Dialecte non vu (F2) : chute mesurée. Combien de tweets maghrébins (magreb) dans train et dans test ? Comment évalueriez-vous honnêtement ce modèle avant un déploiement sur des tweets marocains ?
- **Q12.** Analyse d'erreurs (F6) : catégories dominantes, part du bruit d'annotation. Quel modèle mettriez-vous en production, et pourquoi (qualité, robustesse, coût, latence — chiffres à l'appui) ?

7. Livrables & reproductibilité

- **Dépôt Git** : `README.md` (commandes d'exécution), `pyproject.toml` + `uv.lock` (ou `requirements.txt` figé), `src/` (data, preprocess, models, train, evaluate), `notebooks/`, `tests/`, `results/results.csv`, `predictions/`.
- Graines fixées (random, numpy, torch, transformers) ; split imposé ; aucune donnée de test dans train.
- **Prédictions du test** de chaque modèle final : `predictions/<modèle>_seed<k>.csv` (id, y_true, y_pred, probabilités) — l'enseignant recalcule les scores.
- Le compte-rendu ci-après, rempli (6 pages maximum hors annexes).

COMPTE-RENDU — TP 2

À remplir et à rendre (avec le lien du dépôt)

Identité

Binôme ou individuel : INDIVIDUEL / SAKHI SARA

Filière / groupe : MBDIO Date : 18/10/2026

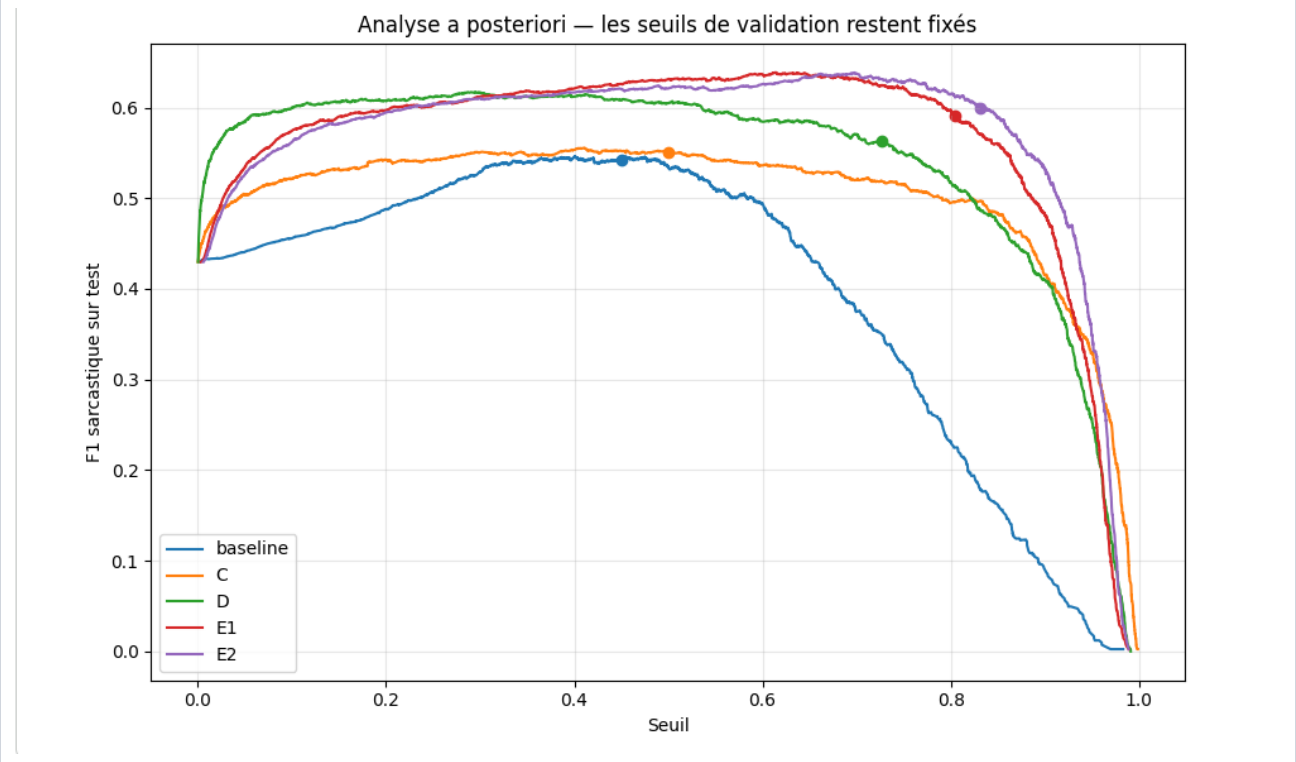
Lien du dépôt : https://github.com/sakhisara/TP2-Sarcasme-Arabe

PLM fine-tunés : AraBERT-twitter et MARBERTv2

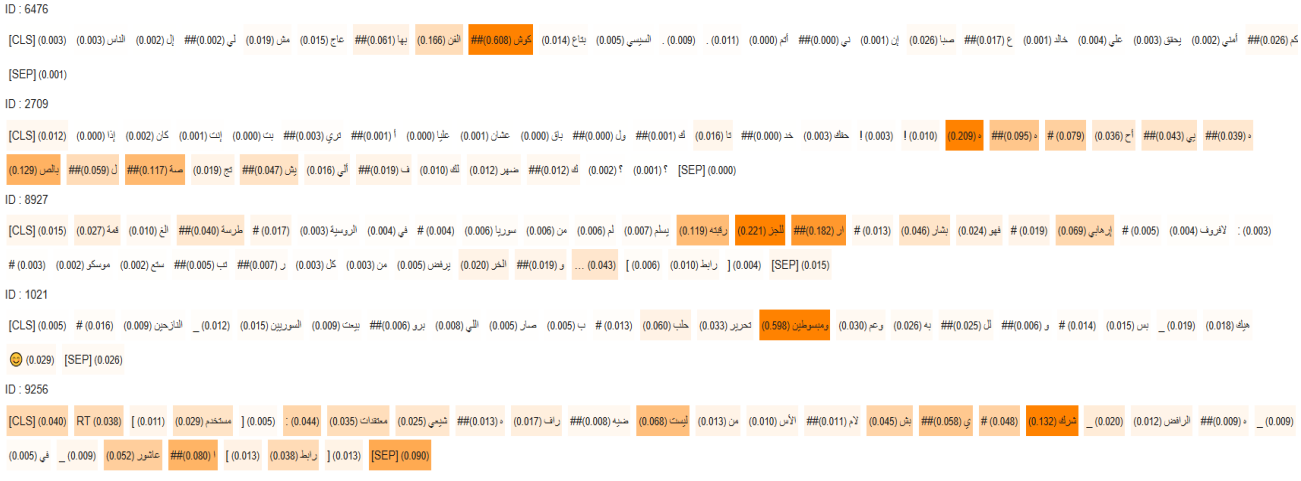
Prétraitement PLM utilisé : **Prétraitement PLM utilisé :** suppression des guillemets externes, nettoyage adapté à chaque modèle et conversion du texte en tokens ; longueur limitée à 64 tokens, avec ajout de padding aux séquences courtes pour traiter les tweets par lots.

Courbe F1 = f(seuil) sur le test (F1) — coller la figure

« Analyse a posteriori de l'effet du seuil sur le F1 test. Les seuils retenus pour l'évaluation finale restent ceux choisis sur validation. »



Attention sur 5 tweets sarcastiques (D2) — coller la figure



Réponses aux questions

Q1. Taux de sarcasme dans train, val et test, globalement et par dialecte. Accuracy et F1 du classifieur « jamais sarcastique ». Quelle métrique retenez-vous, et pourquoi ?

Dialecte	Train	Validation	Test
Égyptien (egypt)	35,06 %	36,15 %	50,65 %
Golfe (gulf)	23,03 %	20,59 %	17,70 %
Levant (levant)	21,48 %	29,09 %	29,79 %
Maghreb (magreb)	37,14 %	0,00 %	0,00 %
Arabe standard (msa)	10,79 %	10,73 %	25,61 %

Les tweets non sarcastiques sont majoritaires : train contient 1 875 sarcastiques sur 10 913 tweets, validation 217 sur 1 255 et test 821 sur 3 000. Train et validation présentent des proportions proches, tandis que le test contient davantage de sarcasme. Les taux varient également selon le dialecte. Les résultats maghrébins sont toutefois peu représentatifs, avec seulement 35 tweets dans train, 5 dans validation et 2 dans test ; les taux nuls ne permettent pas de conclure à l'absence de sarcasme dans ce dialecte.

	modele	pretraitement	graine	seuil	val_accuracy	val_precision_sarc	val_rappel_sarc	val_f1_sarc	val_macro_f1
0	Jamais sarcastique	aucun	42	None	0.827092	0.0	0.0	0.0	0.452682

Sur validation, le classifieur « jamais sarcastique » obtient une accuracy de 82,71 %, mais un F1 sarcastique de 0, puisqu'il ne détecte aucun tweet sarcastique. Nous retenons donc le F1 de la classe sarcastique comme métrique principale : il combine précision et rappel sur la classe à détecter, alors que l'accuracy peut être élevée simplement en prédisant toujours la classe majoritaire.

Q2. Raccourcis : quels indices superficiels (artefact de format, URL, mentions) sont corrélés au label ? Que vaut un classifieur à une seule règle ? Qu'en concluez-vous sur la validité de vos scores ?

Les URL et les guillemets externes sont associés à un taux de sarcasme plus faible dans train : 6,36 % avec URL contre 23,86 % sans URL, et 11,93 % avec guillemets contre 25,30 % sans guillemets. L'association des mentions est faible et change de sens entre train et validation. Les règles « indice absent ⇒ sarcastique » obtiennent sur validation un F1 de 0,362 pour les URL, 0,266 pour les mentions et 0,301 pour les guillemets, malgré leur absence de compréhension du texte.

Ces résultats montrent qu'un modèle peut exploiter des indices superficiels liés à la collecte des données. Les guillemets externes concernent notamment 60,54 % du train brut contre 0,20 % du test, ce qui rend ce raccourci peu transférable. Dans F3, après réentraînement sans URL ni mentions, le F1 test de D passe de 0,5625 à 0,6003, celui d'E2 de 0,5998 à 0,6088, tandis qu'E1 reste presque stable (0,5908 à 0,5916). La suppression de ces indices améliore donc certains modèles, mais pas systématiquement tous. Les scores doivent être interprétés avec les tests de robustesse : ils ne prouvent pas, à eux seuls, une compréhension du sarcasme.

Q3. Quels choix de prétraitement avez-vous faits par famille de modèles ? Lesquels détruisent des indices du sarcasme (ablation 4.3, chiffres à l'appui) ?

Pour TF-IDF et B1/B2, nous avons retiré les guillemets externes et harmonisé les espaces, tout en conservant les emojis, rires, allongements et ponctuations. Pour les réseaux C, nous avons également supprimé les diacritiques et le tatweel, normalisé les variantes d'alif, puis découpé les textes en mots et symboles. Le vocabulaire est construit uniquement sur train, avec une fréquence minimale de deux occurrences ; les tokens inconnus sont remplacés par <UNK>. Pour AraVec, nous appliquons clean_str aux tokens recherchés dans son vocabulaire.

Pour AraBERT en D et E, nous utilisons le préprocesseur officiel ArabertPreprocessor après retrait des guillemets externes. Pour MARBERTv2, nous retirons ces guillemets et harmonisons les espaces. Chaque PLM utilise son propre tokenizer, avec une longueur maximale de 64 tokens et un padding dynamique par lot.

L'expérience F4 est réalisée uniquement sur les vrais positifs initiaux, c'est-à-dire les tweets réellement sarcastiques et correctement prédits sarcastiques. Nous supprimons conjointement les emojis, rires et ponctuations répétées, sans modifier le modèle ni son seuil. Le taux de détection, initialement de 100 % sur cette sélection, passe à 94,25 % pour la baseline, 94,76 % pour C, 92,00 % pour D, 94,00 % pour E1 et 94,46 % pour E2, soit des baisses respectives de 5,75, 5,24, 8,00, 6,00 et 5,54 points. Certains vrais positifs deviennent donc des faux négatifs après suppression des marqueurs. Ces résultats indiquent que ces indices contribuent à certaines prédictions, particulièrement pour D. Toutefois, leur suppression groupée ne permet pas d'isoler l'effet de chaque marqueur ; la réduction des allongements n'a pas été testée ici.

Q4. Zero-shot (B1) : score obtenu et sensibilité à la formulation de l'hypothèse (arabe / français / anglais). Pourquoi un modèle NLI peut-il classer sans exemple d'entraînement ? Quel rôle joue l'attention entre prémisse et hypothèse ?

Sur validation, le modèle zero-shot obtient un F1 sarcastique de 0,4005 avec la formulation arabe, 0,4276 avec la française et 0,4075 avec l'anglaise. Les seuils correspondants, optimisés sur validation, sont 0,7429, 0,8447 et 0,7971. La formulation française donne le meilleur résultat, avec un écart maximal de 2,71 points de F1 par rapport à l'arabe. Le résultat dépend donc de la formulation de l'hypothèse ; les tweets restent en arabe dans les trois expériences.

Le modèle peut classer sans exemples d'entraînement de notre tâche, car il a déjà été entraîné à l'inférence en langage naturel (NLI). Il compare le tweet, appelé prémisse, aux hypothèses « ce tweet est sarcastique » et « ce tweet est sérieux », puis évalue leur compatibilité. L'auto-attention entre les tokens de la prémisse et de l'hypothèse permet de mettre en relation leurs mots et leur contexte pour calculer ces scores. Aucun nouvel entraînement n'est effectué sur ArSarcasm-v2 ; seuls la formulation et le seuil sont choisis sur validation.

Q5. Heuristique « négatif ⇒ sarcastique » (B2) : précision, rappel, F1. Que révèle-t-elle de la relation entre sarcasme et sentiment dans ce corpus ? Son score est-il exempt de contamination ?

L'heuristique « sentiment négatif ⇒ sarcastique » obtient sur validation une précision de 33,21 %, un rappel de 79,72 % et un F1 sarcastique de 0,4688. Elle détecte une grande partie des tweets sarcastiques, mais produit beaucoup de faux positifs : seuls environ un tiers des tweets prédits sarcastiques le sont réellement. Cela suggère une association entre sarcasme et sentiment négatif dans ce corpus, sans équivalence entre les deux : un tweet négatif peut exprimer une critique directe sans sarcasme. Les résultats dépendent également des erreurs du modèle de sentiment.

Le score n'est pas garanti exempt de contamination. Le modèle de sentiment CAMeLBERT a été fine-tuné sur ASTD, ArSAS et SemEval, tandis qu'ArSarcasm-v2 contient des tweets issus de réannotations d'ASTD et de SemEval 2017. Certains tweets évalués pourraient donc avoir déjà été utilisés pendant cet entraînement, avec des labels de sentiment. Ce chevauchement pourrait favoriser les performances ; il reste possible, mais n'a pas été vérifié dans notre expérience.

Q6. Déséquilibre : comparez pondération de la perte, sur-échantillonnage et optimisation du seuil (tableau). Pourquoi le sur-échantillonnage doit-il être fait après la séparation train / val ?

	methode	seuil	val_precision_sarc	val_rappel_sarc	val_f1_sarc	val_macro_f1	val_pr_auc	temps_entrainement_s
0	weighted	0.5000	0.4418	0.6820	0.5362	0.7027	0.5134	21.6300
1	oversampling	0.5000	0.4114	0.6636	0.5079	0.6822	0.5009	10.6766
2	threshold	0.3626	0.4582	0.5806	0.5122	0.6966	0.5055	23.8537

```
Méthode retenue : weighted
```

Sur validation, la perte pondérée obtient le meilleur F1 sarcastique (0,5362) et le meilleur rappel : elle retrouve davantage de tweets sarcastiques en donnant plus de poids aux erreurs sur cette classe. Le sur-échantillonnage, qui présente plus souvent les exemples minoritaires pendant l'entraînement, atteint un F1 de 0,5079. La perte standard avec un seuil optimisé sur validation donne la meilleure précision, mais un rappel plus faible, pour un F1 de 0,5122. Nous retenons donc la perte pondérée pour cette expérience, selon la métrique principale du TP.

Le sur-échantillonnage doit être réalisé après la séparation train/validation et uniquement sur train. Sinon, des copies d'un même tweet pourraient se retrouver dans les deux ensembles, créant une fuite de données et des scores artificiellement élevés. La validation doit conserver ses exemples et sa distribution d'origine.

Q7. C1 vs C2 vs C3 (et C4) : résultats. Les poids d'attention se concentrent-ils sur les marqueurs attendus (emojis, rires, allongements, ponctuation) ? Effet de l'encodage positionnel ?

Le meilleur résultat est obtenu par C1 avec AraVec dégelés, avec un F1 de 0,4930. L'attention de C2 améliore les résultats avec les embeddings aléatoires (0,4721 → 0,4898) et AraVec figés (0,3681 → 0,4566), mais les dégrade avec AraVec dégelés (0,4930 → 0,4052). Son ajout n'apporte donc pas une amélioration systématique. C4 ne figure pas dans ce tableau.

Les visualisations de C2, montrent que certains marqueurs attirent l'attention : plusieurs points d'exclamation du tweet 2709 reçoivent chacun un poids proche de 0,06, contre 0,0476 pour le rire et 0,0181 pour le mot allongé. Dans le tweet 1021, l'emoji reçoit un poids de 0,0806, parmi les plus élevés du tweet. L'attention prend donc en compte certains marqueurs attendus, mais les rires et allongements ne dominent pas toujours. Ces exemples ne suffisent pas à généraliser ce comportement ni à expliquer avec certitude les prédictions.

Pour C3, supprimer les positions améliore légèrement le F1 avec les embeddings aléatoires (0,4483 → 0,4522) et davantage avec AraVec figés (0,4417 → 0,4734). En revanche, cela dégrade le résultat avec AraVec dégelés (0,4651 → 0,4434). L'effet de l'encodage positionnel dépend donc de la configuration dans cette exécution, sans permettre de conclure que l'ordre des mots est inutile.

Q8. PLM gelé (D) vs fine-tuné (E1) : gain et coût (temps, paramètres entraînés). AraBERT-twitter vs MARBERT : lequel, et pourquoi ?

Sur validation, avec la graine 42, AraBERT-twitter gelé obtient un F1 sarcastique de 0,6062 avec [CLS], de 0,6247 avec la moyenne des tokens et de 0,6538 avec la tête BiLSTM-attention. Les entraînements durent respectivement 8,92 s, 6,41 s et 23,53 s, avec 769 paramètres entraînés pour les régressions logistiques et 435 586 pour la tête BiLSTM-attention. Il faut ajouter 31,93 s d'extraction, commune aux deux variantes D1, et 33,35 s pour D2.

Le fine-tuning d'AraBERT-twitter atteint un F1 de 0,6875, soit un gain de 6,28 points par rapport à D1 avec moyenne et de 3,37 points par rapport à D2. Ce gain augmente le coût d'apprentissage : 217,12 s d'entraînement et 134,6 millions de paramètres ajustés, puisque l'encodeur apprend également.

Nous retenons AraBERT-twitter pour E1 : MARBERTv2 obtient un F1 légèrement inférieur (0,6861), avec un entraînement plus long (320,14 s) et davantage de paramètres entraînés (162,3 millions). L'écart de F1 reste toutefois faible, 0,14 point, et ne démontre pas une supériorité générale d'AraBERT-twitter.

Q9. Multi-tâche (E2) : l'ajout du sentiment améliore-t-il la détection du sarcasme ? Effet de λ. Pourquoi une tâche auxiliaire peut-elle aider… ou nuire ?

Sur validation, avec la graine 42, l'ajout du sentiment améliore légèrement AraBERT-twitter : le F1 sarcastique passe de 0,6875 en mono-tâche à 0,6932 en multi-tâche avec λ = 0,3, soit un gain de 0,57 point. Sur trois graines, E1 obtient 0,6865 ± 0,0009, contre 0,6946 ± 0,0018 pour E2, soit un gain moyen de 0,81 point.

Le coefficient λ contrôle l'importance de la tâche sentiment dans la perte totale. Pour AraBERT, les F1 obtenus avec λ = 0,3 ; 0,5 ; 1 sont respectivement 0,6932 ; 0,6895 ; 0,6930. Nous retenons donc λ = 0,3 sur validation, même si son résultat est très proche de celui de λ = 1. Pour MARBERTv2, les scores diminuent légèrement lorsque λ augmente : 0,6850 ; 0,6848 ; 0,6811. Ils restent inférieurs au résultat mono-tâche de 0,6861. Le multi-tâche n'améliore donc pas systématiquement les performances.

Apprendre aussi le sentiment peut aider l'encodeur à mieux représenter le texte et améliorer la détection du sarcasme. Cependant, les deux tâches peuvent demander des informations différentes. Si le poids λ est trop élevé, le modèle peut privilégier l'apprentissage du sentiment au détriment du sarcasme. La prédiction du sentiment n'est pas utilisée directement pour décider si le tweet est sarcastique.

Q10. Décalage de prior (F1) : écart entre le seuil choisi sur val et le seuil optimal a posteriori sur le test. Que mettriez-vous en place en production (recalibration, monitoring) ?

	modele	seuil_val_conserve	seuil_test_optimal_analyse_uniquement	ecart_test_moins_val	F1_test_au_seuil_val	F1_test_optimal_aposteriori
0	baseline	0.4498	0.4005	-0.0493	0.5424	0.5464
1	C	0.5000	0.4109	-0.0891	0.5499	0.5556
2	D	0.7260	0.2945	-0.4316	0.5625	0.6172
3	E1	0.8041	0.6141	-0.1900	0.5908	0.6388
4	E2	0.8317	0.6976	-0.1342	0.5998	0.6386

Le taux de sarcasme passe de 17,29 % sur validation à 27,37 % sur test, ce qui indique un décalage de proportion entre les classes. Dans le tableau, les seuils optimaux calculés a posteriori sur test sont tous inférieurs aux seuils choisis sur validation. Les écarts sont de −0,0493 pour la baseline, −0,0891 pour C, −0,4316 pour D, −0,1900 pour E1 et −0,1342 pour E2. D présente le décalage de seuil le plus important.

Pour E2, le seuil passe de 0,8317 à 0,6976, et le F1 test de 0,5998 à 0,6386. Cependant, le second seuil a été choisi en connaissant les vrais labels du test : ce résultat sert uniquement à l'analyse. La performance à retenir reste 0,5998 au seuil fixé sur validation. Le changement de proportion peut contribuer à ces écarts, mais il ne suffit pas à expliquer à lui seul toutes les différences observées.

En production, nous surveillerions la distribution des scores, la proportion de prédictions sarcastiques et les performances par dialecte. Un échantillon récent régulièrement annoté permettrait de suivre la précision, le rappel et le F1. Si une dérive est confirmée, nous recalibrerions les probabilités et choisirions un nouveau seuil sur des données récentes représentatives, puis vérifierions son efficacité sur un ensemble indépendant.

Q11. Dialecte non vu (F2) : chute mesurée. Combien de tweets maghrébins (magreb) dans train et dans test ? Comment évalueriez-vous honnêtement ce modèle avant un déploiement sur des tweets marocains ?

```
===== DIALECT =====
```

	ensemble	regroupement	groupe	effectif	sarcastiques	taux_sarcasme_pct
1	train	dialect	egypt	2330	817	35.06
2	train	dialect	gulf	521	120	23.03
3	train	dialect	levant	554	119	21.48
4	train	dialect	magreb	35	13	37.14
22	test	dialect	magreb	2	0	0.00

Sur les 306 tweets égyptiens du test, E2 obtient un F1 sarcastique de 0,7829 lorsqu'il est entraîné sur tout train, contre 0,7771 après retrait des tweets égyptiens, soit une baisse d'environ 0,57 point. La chute est faible dans cette expérience, avec des seuils choisis séparément sur validation (0,8317 et 0,5329). Elle ne garantit cependant pas une bonne généralisation à d'autres dialectes.

Le corpus contient seulement 35 tweets magreb dans train après nettoyage et 2 dans test, dont aucun n'est étiqueté sarcastique dans le test. Ces données sont donc insuffisantes pour évaluer honnêtement la détection du sarcasme marocain. De plus, magreb regroupe plusieurs usages maghrébins et ne représente pas exclusivement la darija marocaine.

Avant un déploiement, il faudrait constituer un corpus marocain représentatif, annoté par des personnes maîtrisant ces usages, choisir le seuil sur une validation dédiée et mesurer le F1, la précision et le rappel sur un test indépendant.

Q12. Analyse d'erreurs (F6) : catégories dominantes, part du bruit d'annotation. Quel modèle mettriez-vous en production, et pourquoi (qualité, robustesse, coût, latence — chiffres à l'appui) ?

L'analyse des 30 erreurs d'E2 relève 9 cas liés au contexte (30 %), 8 cas de bruit d'annotation suspecté (26,67 %), 5 cas d'ironie sans marqueur, 5 de marqueurs mal interprétés et 3 liés au dialecte. Aucun ne correspond aux doublons contradictoires retrouvés : les huit labels suspects restent donc non confirmés. Ces proportions concernent uniquement cet échantillon.

Nous retiendrions E2 pour un déploiement pilote, avec son seuil choisi sur validation. Il obtient le meilleur F1 test moyen : 0,6094 ± 0,0094, pour 291,45 s d'entraînement, 513,49 Mio et 107,27 ms par tweet sur CPU. Sa robustesse reste toutefois limitée : le F1 atteint 0,0833 avec URL contre 0,6075 sans URL. Un suivi par sous-groupe serait nécessaire, et la latence, mesurée sur seulement 30 tweets, devrait être confirmée en conditions réelles.

Décision & justification

Modèle retenu pour la production, seuil de décision et pourquoi (qualité / robustesse / coût / latence, chiffres à l'appui) :

Nous choisissons E2, AraBERT-twitter multi-tâche avec $\lambda = 0,3$, car il obtient le meilleur F1 moyen sur test : 0,6094 sur trois graines. Pendant l'entraînement, il apprend à reconnaître le sarcasme et le sentiment. Le sentiment sert de tâche supplémentaire pour aider le modèle à comprendre les tweets.

Pour le modèle de la graine 42, nous conservons le seuil de 0,8317 choisi sur validation : si le score de sarcasme est supérieur ou égal à ce seuil, le tweet est classé sarcastique. Ce modèle obtient un F1 test de 0,5998.

Son entraînement prend en moyenne 291 secondes, il occupe 513,49 Mio et nécessite environ 107 ms par tweet sur CPU. Ce coût peut convenir à un usage à faible volume, mais la vitesse doit être vérifiée en conditions réelles, car elle a été mesurée sur seulement 30 tweets.

Enfin, le modèle reste fragile sur certains textes : son F1 est de 0,0833 avec URL contre 0,6075 sans URL. Nous proposons donc un essai à petite échelle, avec un suivi des erreurs et des performances selon les types de tweets, avant une utilisation plus large.

Limites & pistes d'amélioration

Les principales limites sont le déséquilibre des classes, la différence de proportion de sarcasme entre validation et test, et le manque de tweets maghrébins pour évaluer la darija marocaine. Le modèle rencontre aussi des difficultés avec les tweets contenant des URL et ceux nécessitant un contexte culturel ou politique. Certains labels paraissent discutables, et l’analyse de 30 erreurs reste limitée.

Pour améliorer le système, nous proposons d’enrichir les données avec des tweets dialectaux variés, notamment marocains, de faire vérifier les annotations ambiguës et d’approfondir les tests sur les URL, emojis et rires. Le seuil pourrait être adapté sur une nouvelle validation représentative des usages réels. Enfin, il faudrait mesurer la latence sur davantage de tweets et étudier la compression du modèle pour réduire son coût d’utilisation.